19 және 20 практикалық жұмыстардың есебі

Бағдарламалық қамтамасыз ету конфигурациясы мен нұсқаларын басқару және CI процестерін ұйымдастыру

Орындалған күні 3 қараша 2026 жыл

Барлық 9 тапсырма орындалды. Жергілікті тексеруде 134 тест өтті, Ruff қате таппады. GitHub Actions Python 3.11 және 3.12 орталарында іске қосылды. Екі pull request біріктірілді, v1.0.0 бойынша пакет құрастырылды.

Мақсат — код нұсқасын ғана сақтау емес, өзгерістің себебін, мақұлдауын, бекітілген күйін және тексеру дәлелдерін байланыстыру. Әр тапсырманың төмендегі бөлімінде алгоритм, сәулет, код үзіндісі, нақты нәтиже және қорғау түсіндірмесі берілген.

Нәтижелер тізімі

№	Жоба	Тест	Коммит
1	file-version-manager	12	15
2	change-tracker	9	15
3	configuration-scanner	13	15
4	task-manager	11	20
5	warehouse-ci	16	16
6	release-manager	18	17
7	configuration-auditor	16	16
8	scm-platform	25	25
9	scm-traceability	14	18

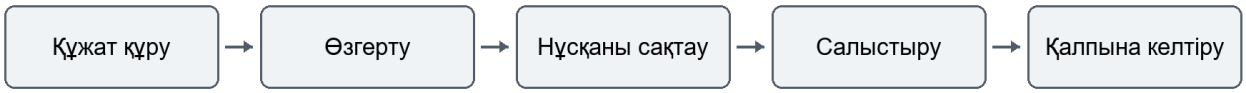
Репозиторий https://github.com/Aldyar7122/AldyarLab19

Жұмыс үстеліндегі Практика_19_20 қалтасында тоғыз жеке Git репозиторийі бар. GitHub_AldyarLab19 ішіндегі ортақ репозиторий оларды projects каталогында біріктіреді. Жеке тарихтар Git bundle файлдарында да сақталған.

1 тапсырма Мәтіндік құжаттар нұсқаларының менеджері

Мақсат Мәтіндік құжаттың бірнеше күйін сақтау, салыстыру және бұрынғы күйді тарихты жоғалтпай қалпына келтіру.

Сәулет Console CLI → Manager → әр құжатқа жеке JSON файл. UUID нұсқаны, UTC уақыты сақталған сәтті анықтайды. difflib жолдардағы өзгерістерді көрсетеді.



Орындалған жұмыс Құру, түзету, сақтау, нұсқалар тізімі, unified diff және restore әрекеттері іске асырылды. Қате атау, жоқ нұсқа және қайталанған құжат тексеріледі.

Кодтың негізгі үзіндісі

```
def compare(self, name, old, new):
    a = self._version(name, old)["content"].splitlines()
    b = self._version(name, new)["content"].splitlines()
    return {"diff": "\n".join(difflib.unified_diff(a, b, old, new, lineterm=""))}
```

Нақты нәтиже Бірінші күйдегі Old sentence жолы жойылып, New sentence және CI added жолдары қосылғаны көрсетілді. Бірінші нұсқа қалпына келтірілгеннен кейін тарихта 3 нұсқа қалды.

Тексеру 12 pytest тесті өтті. Ruff нәтижесі All checks passed. Git тарихында 15 коммит, 4 тармақ және v1.0.0 тегі бар. Дәлелдер жоба ішіндегі evidence каталогында сақталған.

Қорғауда айту Қалпына келтіру ескі жазбаны өшірмейді. Таңдалған мәтін ағымдағы мазмұнға көшіріліп, жаңа нұсқа ретінде сақталады. Сондықтан бұрынғы екі күй де тексеруге қолжетімді.

Шектеу Жергілікті UTF-8 мәтінмен жұмыс істейді; бірнеше процестің бір JSON файлға қатар жазуын үйлестіретін құлып жоқ.

Қайта көрсету: python demo.py. Консоль параметрлері API.md ішінде. Нақты нәтижені evidence/demo.json арқылы ашуға болады.

2 тапсырма Жобаның өзгерістер журналы

Мақсат Өзгеріске сұрауларды тіркеу және олардың шешімін Git коммиттерімен байланыстырып қадағалау.

Сәулет Console CLI → Tracker → SQLite changes және history кестелері → JSON есеп. ID бірегейлігі дерекқорда да бақыланады.



Орындалған жұмыс ID, автор, күн, түр, сипаттама, мәртебе және commit_hash сақталады. Автор, түр және мәртебе бойынша бір мезгілде сүзгілеуге болады. Әр мәртебе ауысуы history кестесіне жазылады.

Кодтың негізгі үзіндісі

```
def transition(self, id, status, hash=None):
    rows = self.list(id=id)
    if not rows:
        raise ValueError("unknown change ID")
    allowed = {"planned": "in_progress", "in_progress": "completed"}
    if allowed.get(rows[0]["status"]) != status:
        raise ValueError("invalid status transition")
```

Нақты нәтиже CR-001 және CR-002 сұраулары репозиторийдегі екі нақты толық Git hash-пен байланыстырылды. Екеуі completed күйіне өтті; есепте 2 сұрау және 6 мәртебе оқиғасы бар.

Тексеру 9 pytest тесті өтті. Ruff нәтижесі All checks passed. Git тарихында 15 коммит, 4 тармақ және v1.0.0 тегі бар. Дәлелдер жоба ішіндегі evidence каталогында сақталған.

Қорғауда айту planned күйінен бірден completed күйіне өтуге болмайды. Рұқсат етілген тізбек қана қабылданады. Аяқталған өзгерістегі commit_hash оның кодтағы дәлелін табуға мүмкіндік береді.

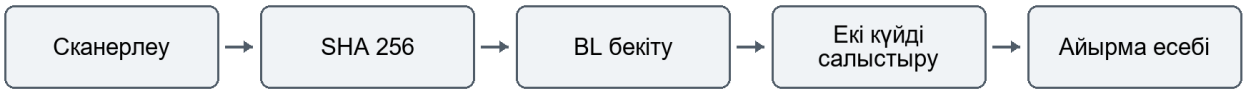
Шектеу Коммит hash-ының пішімі тексеріледі; оның репозиторийде бар-жоғы demo мен Git журналы арқылы бөлек расталады.

Қайта көрсету: python demo.py. Консоль параметрлері API.md ішінде. Нақты нәтижені evidence/demo.json арқылы ашуға болады.

3 тапсырма Жоба файлдарының конфигурациясын бақылау

Мақсат Файлдардың бекітілген күйін тіркеп, жаңа күйдегі қосылған, жойылған және өзгерген файлдарды анықтау.

Сәулет Console CLI → Scanner → рекурсивті Path сканері және SHA-256 → .scm ішіндегі JSON baseline және history.jsonl.



Орындалған жұмыс Салыстырмалы жол, көлем, CI ID, мазмұн нұсқасы және SHA-256 есептеледі. .git, __pycache__, venv, .venv, .scm және пайдаланушы ережелері алынып тасталады. Жаңа baseline алдыңғысына сілтеме жасайды.

Кодтың негізгі үзіндісі

```
try:
    with path.open("x", encoding="utf-8") as out:
        json.dump(data, out, ensure_ascii=False, indent=2)
except FileExistsError as exc:
    raise ValueError("approved baseline is immutable; use a new ID") from exc
return data
```

Нақты нәтиже BL-001 және BL-002 құрылды. app.txt өзгерді, obsolete.txt жойылды, new.txt қосылды. differences.json осы үш санатты нақты тізімдейді. Snapshot файлдары өз сканерлеуіне кірмейді.

Тексеру 13 pytest тесті өтті. Ruff нәтижесі All checks passed. Git тарихында 15 коммит, 4 тармақ және v1.0.0 тегі бар. Дәлелдер жоба ішіндегі evidence каталогында сақталған.

Қорғауда айту Файлдың атауы бірдей болғанымен, SHA-256 өзгерсе, мазмұны өзгерген деп есептеледі. Baseline файл x режимімен жасалады: бұрынғы ID-ге қайта жазу тоқтатылады.

Шектеу Қорғаныс қолданба деңгейінде орындалады; файлға сырттан тікелей қолжетімді адамды операциялық жүйе құқықтары арқылы шектеу қажет.

Қайта көрсету: python demo.py. Консоль параметрлері API.md ішінде. Нақты нәтижені evidence/demo.json арқылы ашуға болады.

4 тапсырма Git тапсырмалары мен қақтығыстарын басқару

Мақсат SQLite тапсырмалар менеджерін әзірлеу және екі тәуелсіз тармақтың нақты Git қақтығысын функцияларды жоғалтпай шешу.

Сәулет Console CLI → Tasks → SQLite tasks кестесі. conflict_demo.py ортақ функциялар тізілімі ретінде екі тармақта өзгертілді.



Орындалған жұмыс CRUD, орындаушы, басымдық, мәртебе, күн, үш өрісті сүзгі және орындалу статистикасы қосылды. feature-tasks, feature-filter, feature-statistics тармақтары сақталған.

Кодтың негізгі үзіндісі

```
def statistics(self):
    rows = self.list()
    done = sum(row["status"] == "done" for row in rows)
    return {"total": len(rows), "done": done, "completion_percent": round(done / len(rows) *
    100, 2) if rows else 0}
```

Нақты нәтиже Көрсетілімде 2 тапсырманың екеуі де аяқталды: 100%. Қақтығыстан кейін features() нәтижесі tasks, filter, statistics болды. Қақтығыс белгілері, екі бастапқы hash және қорытынды merge hash evidence ішінде бар.

Тексеру 11 pytest тесті өтті. Ruff нәтижесі All checks passed. Git тарихында 20 коммит, 6 тармақ және v1.0.0 тегі бар. Дәлелдер жоба ішіндегі evidence каталогында сақталған.

Қорғауда айту Git бір файлдағы бір жолдың екі түрлі нұсқасын автоматты таңдай алмады. Шешімде filter мен statistics екеуі де сақталды; merge commit екі ата-ананы байланыстырады, тест екі функцияның жұмысын растайды.

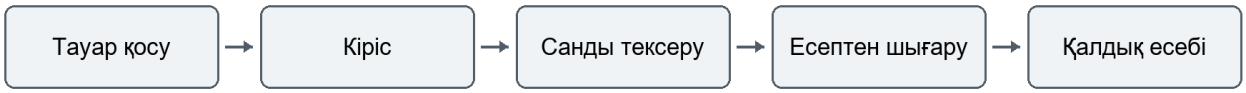
Шектеу Қақтығыс оқуға арналған шағын файлда әдейі жасалды; бұл нақты Git merge қақтығысы, мәтіндік имитация емес.

Қайта көрсету: git log --all --graph --oneline. Консоль параметрлері API.md ішінде. Нақты нәтижені evidence/demo.json арқылы ашуға болады.

5 тапсырма Үздіксіз интеграциясы бар қойма жүйесі

Мақсат Қойма тауарларын, кірістер мен есептен шығаруды есепке алу және pytest пен ruff тексерулерін GitHub Actions арқылы іске қосу.

Сәулет Console CLI → Warehouse → SQLite products және operations кестелері. Қалдық пен операция журналы бір транзакцияда жаңартылады.



Орындалған жұмыс Тауар CRUD, санат, іздеу, қалдық есебі, операция журналы және теріс қалдыққа тыйым салу орындалды. Жойылған тауар архивтеліп, бұрынғы операциялары сақталады.

Кодтың негізгі үзіндісі

```
def move(self, sku, kind, quantity):
    positive(quantity)
    if kind not in {"receipt", "writeoff"}:
        raise ValueError("operation must be receipt or writeoff")
    self.get(sku)
    delta = quantity if kind == "receipt" else -quantity
    with self.db:
```

Нақты нәтиже NB-001 тауарына 20 дана кіріс жасалып, 7 дана есептен шығарылды: қалдық 13. 100 дананы есептен шығару insufficient stock қатесімен тоқтатылды. GitHub-та қате сан тексеруін алып тастаған коммит CI-ден өтпеді; түзету CI-ден өтті.

Тексеру 16 pytest тесті өтті. Ruff нәтижесі All checks passed. Git тарихында 16 коммит, 4 тармақ және v1.0.0 тегі бар. Дәлелдер жоба ішіндегі evidence каталогында сақталған.

Қорғауда айту UPDATE шартындағы stock + delta >= 0 тексеруі теріс қалдықты тоқтатады. Қате болса, транзакция кері қайтады және операция журналына жалған жазба қосылмайды.

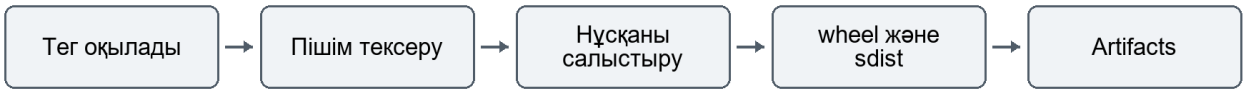
Шектеу Саны бүтін бірлікпен есептеледі; бөлшек өлшем, баға және көп қоймалы есеп осы нұсқада қамтылмаған.

Қайта көрсету: python demo.py. Консоль параметрлері API.md ішінде. Нақты нәтижені evidence/demo.json арқылы ашуға болады.

6 тапсырма Бағдарламалық шығарылымдарды автоматтандыру

Мақсат Пакет нұсқаларын MAJOR.MINOR.PATCH ережесімен басқару және сәйкес Git тегі бойынша wheel мен sdist құрастыру.

Сәулет Console CLI → Releases → pyproject.toml, releases.json және CHANGELOG.md. CI мен тег бойынша Release workflow бөлек орналасқан.



Орындалған жұмыс Қатаң сандық SemVer, major/minor/patch арттыру, төменгі бөліктерді нөлдеу, кері нұсқа мен қайталанған шығарылымға тыйым салу іске асырылды. Нұсқа мен күн өзгерістер журналына жазылады.

Кодтың негізгі үзіндісі

```
def check_tag(self, tag):
    if not tag or not isinstance(tag, str) or not tag.startswith("v"):
        raise ValueError("release requires a vMAJOR.MINOR.PATCH Git tag")
    parse(tag[1:])
    if tag[1:] != self.current():
        raise ValueError(f"tag {tag} differs from package version {self.current()}")
    return {"tag": tag, "version": self.current(), "valid": True}
```

Нақты нәтиже Көрсетілімде 1.0.0 → 1.0.1 → 1.1.0 → 2.0.0 алынды. Негізгі пакет 1.0.0 күйінде қалды. v1.0.0 сәйкес келді; v9.9.9 тоқтатылды. wheel және sdist жасалып, wheel жеке каталогқа орнатылып импортпен тексерілді.

Тексеру 18 pytest тесті өтті. Ruff нәтижесі All checks passed. Git тарихында 17 коммит, 4 тармақ және v1.0.0 тегі бар. Дәлелдер жоба ішіндегі evidence каталогында сақталған.

Қорғауда айту Әдеттегі push және pull_request кезінде тесттерге тег қажет емес. Тек release workflow тегті пакет нұсқасымен салыстырады. Тексеру сәтсіз болса, build қадамына жетпейді және артефакт жасалмайды.

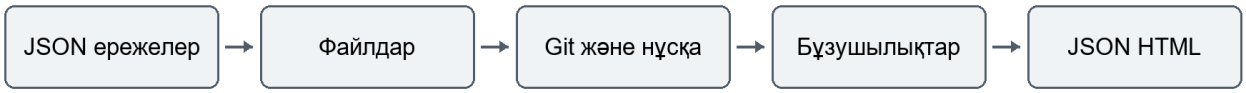
Шектеу Тапсырмадағы сандық MAJOR.MINOR.PATCH ішкі жиыны іске асырылған; SemVer prerelease және build metadata жұрнақтары қабылданбайды. Пакет PyPI-ге жарияланбайды.

Қайта көрсету: python -m pip install dist/*.whl. Консоль параметрлері API.md ішінде. Нақты нәтижені evidence/demo.json арқылы ашуға болады.

7 тапсырма Бағдарламалық жоба конфигурациясының аудиті

Мақсат Жоба ішкі ережелерінің сақталуын бапталатын құралмен тексеріп, маңыздылық деңгейі бар JSON және HTML есептерін шығару.

Сәулет Console CLI → Auditor → JSON rules → файл, TOML, Git және workflow тексерулері → findings тізімі және екі есеп пішімі.



Орындалған жұмыс README, CHANGELOG, pyproject, Git, CI, нұсқа пішімі және ең аз коммит саны тексеріледі. Зақымдалған JSON/TOML және жоқ файлдар апатты аяқталусыз қате қорытындысына айналады.

Кодтың негізгі үзіндісі

```
def audit(self, rules=None, output=None):
    findings = []

    def issue(rule, severity, message):
        findings.append({"rule": rule, "severity": severity, "message": message})
```

Нақты нәтиже Сәйкес жоба passed=true нәтижесін алды. Арнайы сәйкес емес жобада README, CHANGELOG, CI және Git жоқтығы, сондай-ақ 01.0.0 нұсқасының жарамсыздығы анықталды. Екі жобаға JSON және HTML есептері құрылды.

Тексеру 16 pytest тесті өтті. Ruff нәтижесі All checks passed. Git тарихында 16 коммит, 4 тармақ және v1.0.0 тегі бар. Дәлелдер жоба ішіндегі evidence каталогында сақталған.

Қорғауда айту Функционалдық тест нақты функцияның дұрыстығын тексереді. Аудит жоба құрылымы мен басқару ережелерінің орындалуын тексереді. Аудит сәтсіз болса, CLI 1 кодын қайтарады, сондықтан CI тоқтайды.

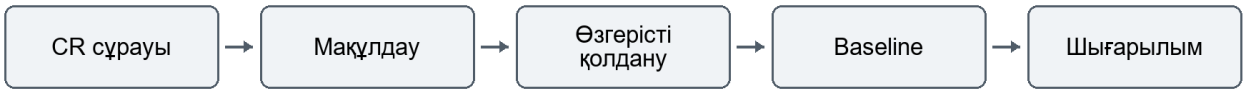
Шектеу Workflow құрылымы негізгі jobs және steps белгілерімен тексеріледі; толық YAML семантикасы мен тәуелсіз стандарт аудиті бағаланбайды.

Қайта көрсету: python demo.py. Консоль параметрлері API.md ішінде. Нақты нәтижені evidence/demo.json арқылы ашуға болады.

8 тапсырма Кешенді SCM платформасы

Мақсат Бірнеше жобаның бірліктерін, өзгеріс сұрауларын, baseline және шығарылымдарын бір SQLite жүйесінде басқару.

Сәулет Console CLI → Platform → projects, items, item_versions, requests, baselines, releases және history кестелері. Әр өзгеріс транзакцияда тіркеледі.



Орындалған жұмыс Жобалар мен CI нұсқалары, шешім себебі, мақұлдаушы, commit hash, тест сілтемесі және оқиғалар сақталады. Мақұлданбаған өзгеріске тыйым салынады. Baseline UPDATE және DELETE әрекеттері SQLite триггерімен тоқтатылады.

Кодтың негізгі үзіндісі

```
def apply(self, id, commit, test):
    rows = self.requests(id)
    if not rows or rows[0]["status"] != "approved":
        raise ValueError("only an approved request can be applied once")
    row = rows[0]
    commit_hash(commit)
    text(test)
```

Нақты нәтиже P1 және P2 жобалары тіркелді. CR-001 мақұлданғанға дейін қолданылмады; мақұлдаудан кейін CI-001 нұсқасы 1.0-ден 1.1-ге өтті. BL-001/BL-002 айырмасы CI-001 болды. 1.0.0 шығарылымы BL-002-мен байланыстырылды. Екі GitHub pull request жасалып тексерілді.

Тексеру 25 pytest тесті өтті. Ruff нәтижесі All checks passed. Git тарихында 25 коммит, 8 тармақ және v1.0.0 тегі бар. Дәлелдер жоба ішіндегі evidence каталогында сақталған.

Қорғауда айту Git файлдар тарихын сақтайды, ал бұл платформа сол файлға қатысты сұрауды, шешімді және бекітілген конфигурацияны байланыстырады. Мақұлдау apply әрекетінің міндетті шарты болып тұр.

Шектеу Рөлдер оқу тәртібімен бөлінген; пайдаланушы аутентификациясы жоқ. DB әкімшісінің тікелей араласуын криптографиялық түрде шектемейді.

Қайта көрсету: python demo.py. Консоль параметрлері API.md ішінде. Нақты нәтижені evidence/demo.json арқылы ашуға болады.

9 тапсырма ISO IEC IEEE 828 процестерінің қадағалануы

Мақсат Процесс, SCMP ережесі, функция, тест және дәлел арасындағы тексерілетін байланысты құру.

Сәулет Traceability класы SCM тізілімін пайдаланады. Қосымша verify файл SHA-256-ын, chains дәлел байланысын, matrix процесс дәрежесін және audit толықтықты тексереді.



Орындалған жұмыс CI, сұраулар, шешімдер және нұсқалар SQLite-те сақталады. CR → decision → commit → baseline → test тізбегі JSON есебіне кіреді. Матрица JSON және CSV түрінде берілді.

Кодтың негізгі үзіндісі

```
def verify(self, baseline, root):
    base = self.get_baseline(baseline)
    root = Path(root).resolve()
    mismatches = []
    for item in base["items"]:
        path = (root / item["path"]).resolve()
        if not path.is_relative_to(root) or not path.is_file():
```

Нақты нәтиже CR-001 → Approved → нақты Git commit → BL-001 → test_chain → traceability.json тізбегі көрсетілді. Бастапқы SHA-256 тексеруі өтті; main.py сырттан өзгертілген соң сәйкессіздік анықталды. Матрицада 7 процесс пен шектеулер көрсетілді.

Тексеру 14 pytest тесті өтті. Ruff нәтижесі All checks passed. Git тарихында 18 коммит, 4 тармақ және v1.0.0 тегі бар. Дәлелдер жоба ішіндегі evidence каталогында сақталған.

Қорғауда айту Бұл тапсырманың басты нәтижесі жай дерекқор емес, әр талаптың қай ережеге, функцияға, тестке және дәлелге сүйенетінін көрсету. Шектеуі бар процестер partial деп ашық белгіленген.

Шектеу Тест атауы мен commit hash тізілімде сілтеме ретінде сақталады; CI орындалуы GitHub сілтемесімен бөлек дәлелденеді. Толық стандарт сәйкестігі жарияланбайды.

Қайта көрсету: python demo.py. Консоль параметрлері API.md ішінде. Нақты нәтижені evidence/demo.json арқылы ашуға болады.

Git және CI нәтижелерінің дәлелдері

Төмендегі сілтемелер GitHub API арқылы тексерілген нақты іске қосылуларға апарады. Әдейі сәтсіз мысалдар бөлек белгіленген; олардың сәтсіз болуы бақылаудың жұмыс істегенін көрсетеді.

Қоймадағы әдейі қате енгізілген CI: failure.

https://github.com/Aldyar7122/AldyarLab19/actions/runs/41655869782

Санды тексеру қалпына келтірілген CI: success.

https://github.com/Aldyar7122/AldyarLab19/actions/runs/41656030137

Сәйкес емес v9.9.9 тегі: failure.

https://github.com/Aldyar7122/AldyarLab19/actions/runs/41655872432

Сәтті v1.0.0 құрастырылымы: success.

https://github.com/Aldyar7122/AldyarLab19/actions/runs/41657029584

Қорытынды main CI: success.

https://github.com/Aldyar7122/AldyarLab19/actions/runs/41657230512

pull_request оқиғасының сәтті тексеруі: success.

https://github.com/Aldyar7122/AldyarLab19/actions/runs/41656288851

Сәтсіз CI коммиті cf17ce29f47a9a8c8ee24dfb79e6a30da73af809

Түзету коммиті 7e14d0cb016a9a3659f0ace5a5902b73e927319e

Pull request 1 біріктірілді: https://github.com/Aldyar7122/AldyarLab19/pull/1

Pull request 2 біріктірілді: https://github.com/Aldyar7122/AldyarLab19/pull/2

Сәтті release артефактісі release-manager-v1.0.0

Сәйкес емес тег құрастырылымында артефакт саны 0. Сәтті құрастырылымда wheel және sdist GitHub Actions Artifacts ішінде сақталған. Жариялау және deployment орындалмайды.

Қадағалану және басқару шектеулері

SCMP.md құжаттары рөлдерді, CI сәйкестендіруді, сұрауларды келісуді, baseline бекітуді, мәртебелерді есепке алуды, аудит пен шығарылымды сипаттайды. 9 тапсырмадағы matrix.json және matrix.csv әр процестің ережесін, функциясын, тестін және дәлелін байланыстырады.

Процесс	Дәреже	Дәлел
Identification	Іске асырылған	items + item_versions
Change control	Ішінара	requests + history
Baseline	Іске асырылған	baselines + verification.json
Status accounting	Іске асырылған	traceability.json
Audit	Ішінара	audit.json
Release	Ішінара	releases + Git tag
Formal certification	Іске асырылмаған	scope statement

Мақұлдау қолданбада тексеріледі, бірақ оқу рөлдері толық аутентификациямен бөлінбеген. Аудит ішкі ережелерді бағалайды. Git hash және тест атауы тіркелген сілтемелер; олардың нақты орындалу дәлелі Git тарихы мен CI журналында тексеріледі. Сондықтан жұмыс стандартқа толық сәйкестік немесе сертификаттау деп ұсынылмайды.

Қорғауда көрсету реті

1. Осы есептің бірінші бетінен тоғыз жобаның нәтижелерін қысқаша таныстыру.

2. 01 жобада demo.py іске қосып, diff пен 3 нұсқалы тарихты көрсету.

3. 03 жобада екі baseline және differences.json нәтижесін ашу.

4. 04 жобада conflict-before.txt, conflict.json және git log --graph көрсету.

5. GitHub Actions-та сәтсіз CI, түзетілген CI және екі release сценарийін ашу.

6. 08 жобада сұрауды мақұлдағанға дейінгі тыйымды және толық шығарылым сценарийін көрсету.

7. 09 жобада матрицаны, дәлел тізбегін және сырттан өзгертілген файлды анықтау нәтижесін көрсету.

Барлық demo.py стандартты Python кітапханаларын қолданады. Тесттер үшін README-дегі тәуелділіктер орнатылады. Бір командамен көрсетілім үшін түбірдегі DEMO_ALL.ps1 файлын іске қосуға болады.

Бақылау сұрақтарына қысқа жауаптар

1 Конфигурациялық бірлік деген не Басқаруға алынған жеке элемент: код файлы, тест, конфигурация не құжат. Оны CI ID, жол, нұсқа және SHA-256 арқылы сәйкестендіруге болады.

2 Baseline ағымдағы файлдан несімен өзгеше Ағымдағы файл өзгеруі мүмкін. Baseline — бірегей ID, бекітуші, күн және нақты бірліктер нұсқалары тіркелген бақылау күйі. Кейінгі өзгеріс жаңа baseline арқылы өтеді.

3 Өзгерістен бұрын қандай шешім қажет Өзгеріс сұрауы тіркеліп, әсері мен тәуекелі қаралады. Мақұлдау не қабылдамау себебі сақталады. Мақұлданған өзгеріс қана іске асырылады.

4 CR мен Git қадағалануды қалай береді CR өзгерістің себебі мен шешімін, Git commit нақты код өзгерісін көрсетеді. Hash арқылы екеуі байланысады; baseline бекітілген күйді анықтайды.

5 Аудит пен функционалдық тест айырмасы Тест функцияның күтілген нәтижесін тексереді. Конфигурациялық аудит басқарылатын бірліктердің, файлдардың, тарихтың және ішкі ережелердің сәйкестігін тексереді.

6 Branch PR және tag мақсаты Branch өзгерісті бөлек әзірлеуге, PR оны қарау мен біріктіруге, tag белгілі шығарылымды атауға арналған.

7 Қақтығыс қалай расталады Екі тармақ бір жолды әртүрлі өзгертеді. Git CONFLICT көрсетеді. Шешілген файл, merge commit-тің екі ата-анасы және функцияларды тексеретін тест нәтижесі сақталады.

8 pytest және ruff нені тексереді pytest бағдарламаның әрекетін, қателерін және шекаралық жағдайларын орындайды. Ruff кодты іске қоспай статикалық қателер мен белгіленген стиль ережелерін тексереді.

9 CI Delivery Deployment айырмасы CI өзгерісті автоматты тексереді. Continuous Delivery өнімді шығаруға дайын күйге жеткізеді, орналастыру шешімі қолмен қалуы мүмкін. Continuous Deployment тексеруден өткен өзгерісті автоматты орналастырады.

10 Сәтті workflow неге толық сәйкестік емес Workflow тек оған жазылған тексерулерді орындайды. Ол барлық ұйымдық рәсімдерді, тәуелсіз мақұлдауды, аудитті және стандарт талаптарының толық орындалуын автоматты растамайды.

Қорытынды

Нұсқалар тарихы, қадағаланатын сұраулар, өзгермейтін baseline, автоматты тесттер және CI бір басқару тізбегіне біріктірілді. Нәтиже кодтың ағымдағы күйін ғана емес, оның қалай өзгергенін және қалай тексерілгенін де дәлелдеуге мүмкіндік береді.

Негізгі дереккөздер: берілген 19–20 практикалық жұмыс; Git құжаттамасы https://git-scm.com/doc; GitHub Actions https://docs.github.com/en/actions/tutorials/build-and-test-code/python; SemVer https://semver.org/; pytest https://docs.pytest.org/; Ruff https://docs.astral.sh/ruff/.